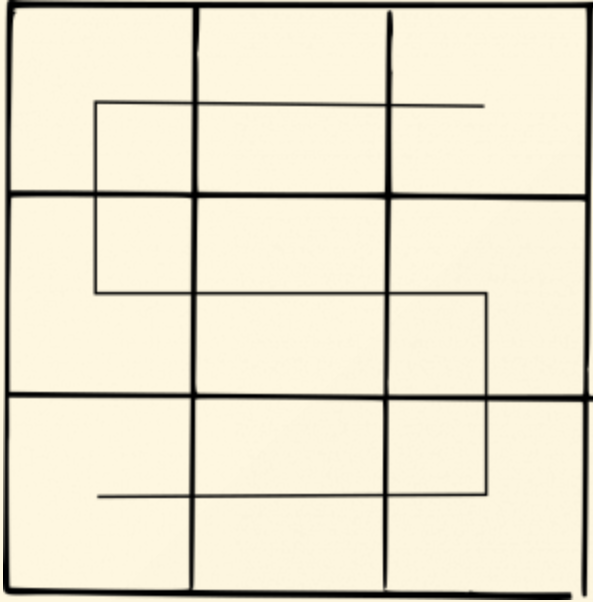
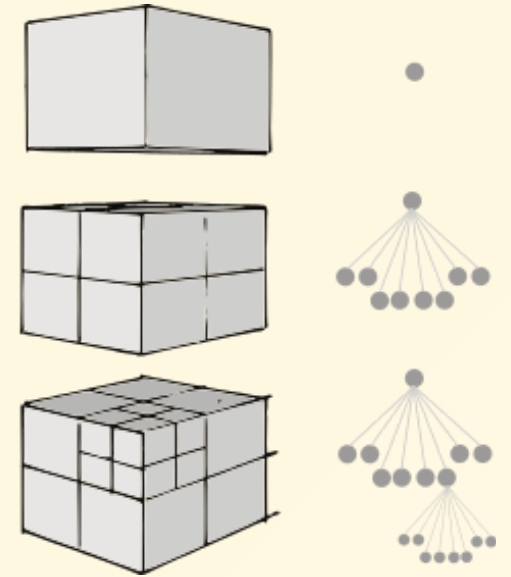




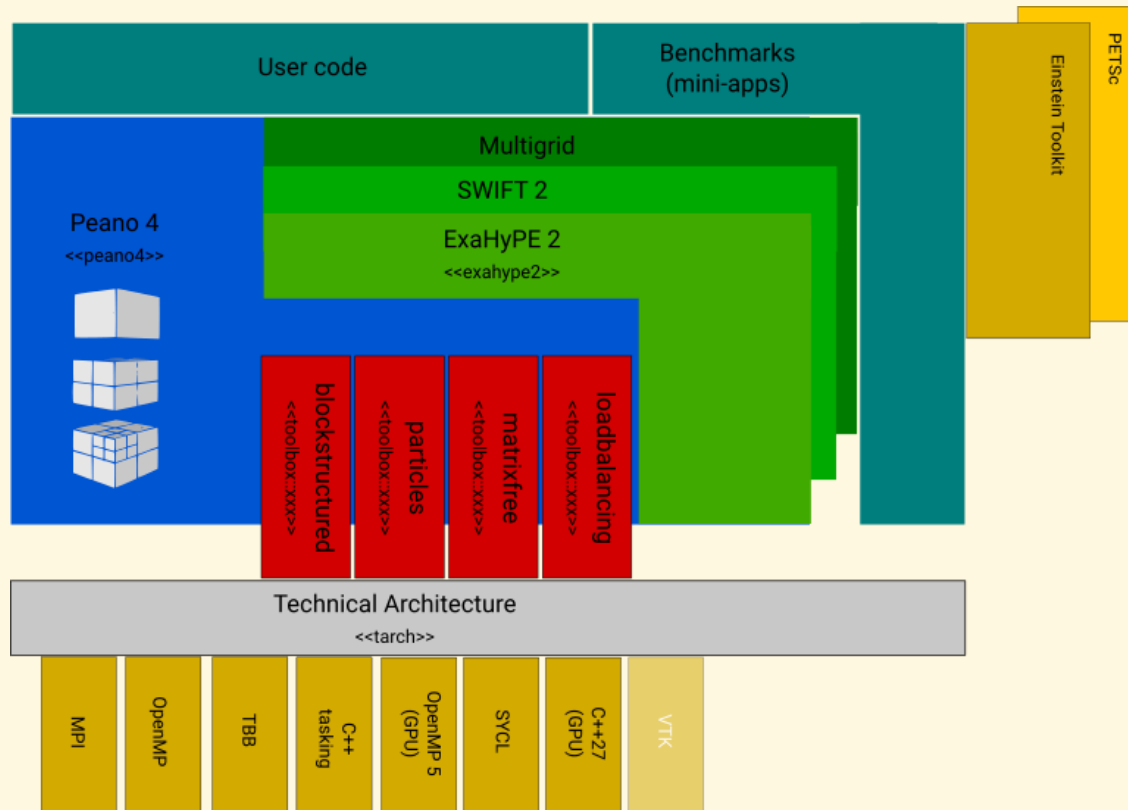
Peano and Load Balancing



Dmitry Nikolaenko | HPC Days 2026



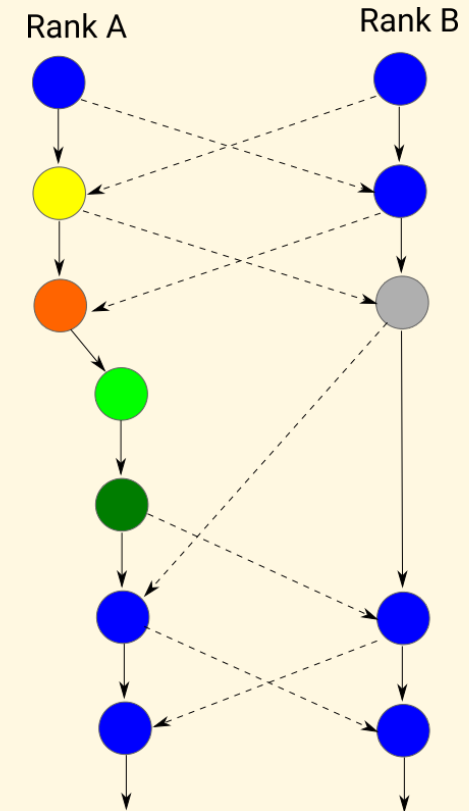
Peano In One Slide



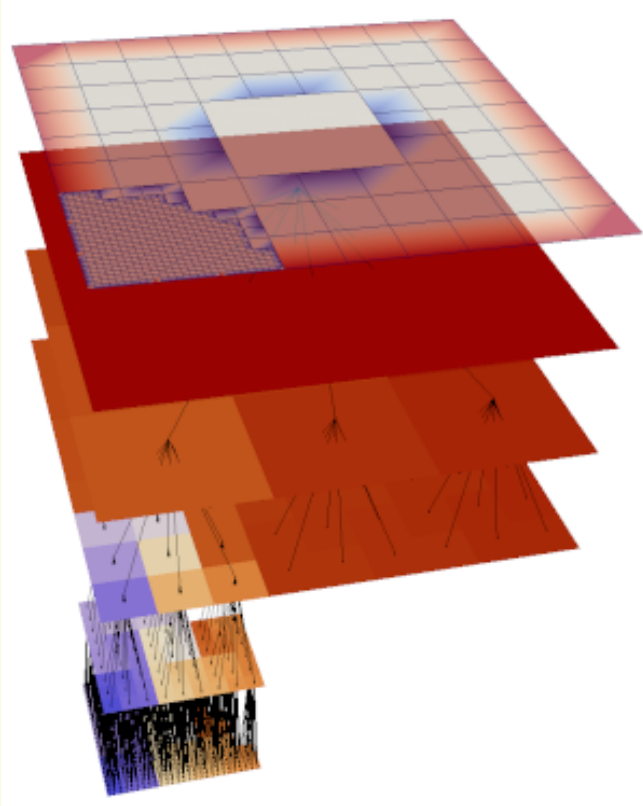
- Peano is the adaptive-grid framework underneath ExaHyPE2 and other toolboxes
- It supports MPI and multithreading through spacetime-based domain decomposition
- Load balancing strategies plug into this stack and steer split decisions

How Load Balancing Works In Peano

- **Strategies** are decision layers; they do not repartition data directly
- Their core job is to **trigger spacetree split recommendations** at the right time
- Practical loop from the toolbox docs:
 1. run **mesh traversals**
 2. periodically invoke the **load balancing strategy**
 3. continue traversals to **realize splits** (and occasional joins)
- The same infrastructure serves MPI and multithreading through tree decomposition



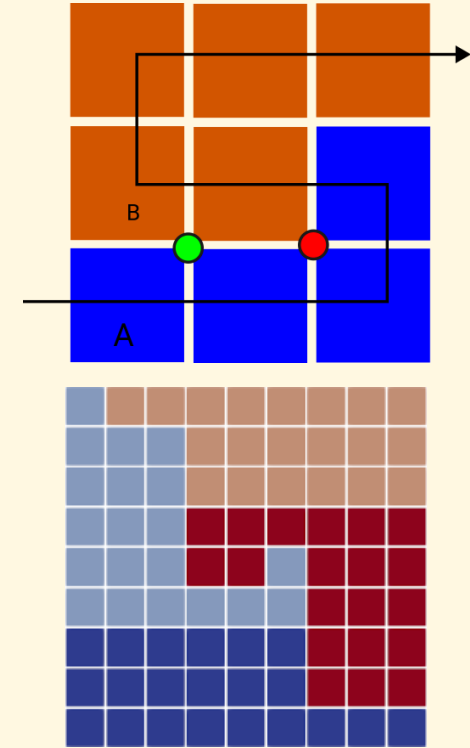
Offline Optimization Framing



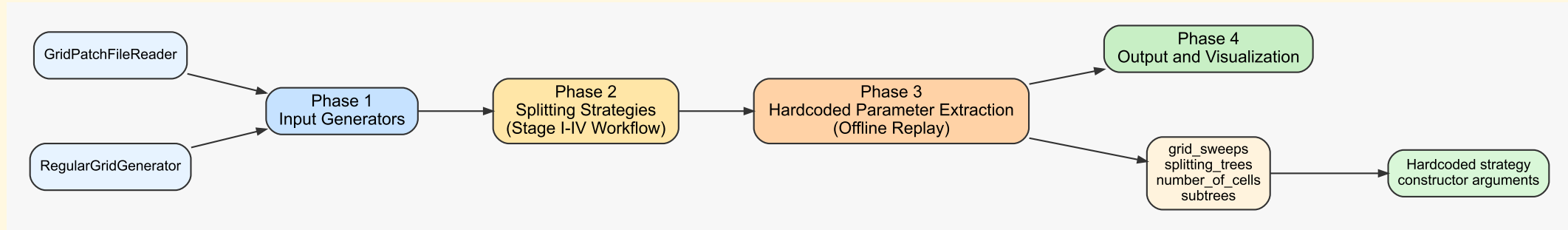
- **Online** splitting acts with incomplete future-tree information
- **Offline** flow uses full-tree knowledge:
 1. build once without balancing
 2. checkpoint and analyze
 3. derive staged split plan
 4. restart with Hardcoded replay
- Objective shifts from only balancing leaves to balancing plus **lower split cost**
- **Subtree ids** from extraction are logical; map them to **destination ranks** at replay time

Domain-Decomposition Constraints

- Strategies do not repartition directly; they issue `split()` recommendations via the **spacetree set**, and ownership cuts are realised along the **Peano space-filling curve (SFC)**
- Peano enforces **topology** and **grid-regularity constraints**, so requested split sizes are approximate
- Decomposition is **tree-based**: splits persist, and joins are limited to degenerated subtrees
- Offline extraction returns **subtree ids** (logical trees); map them to destination **MPI ranks** for replay



Architecture At A Glance



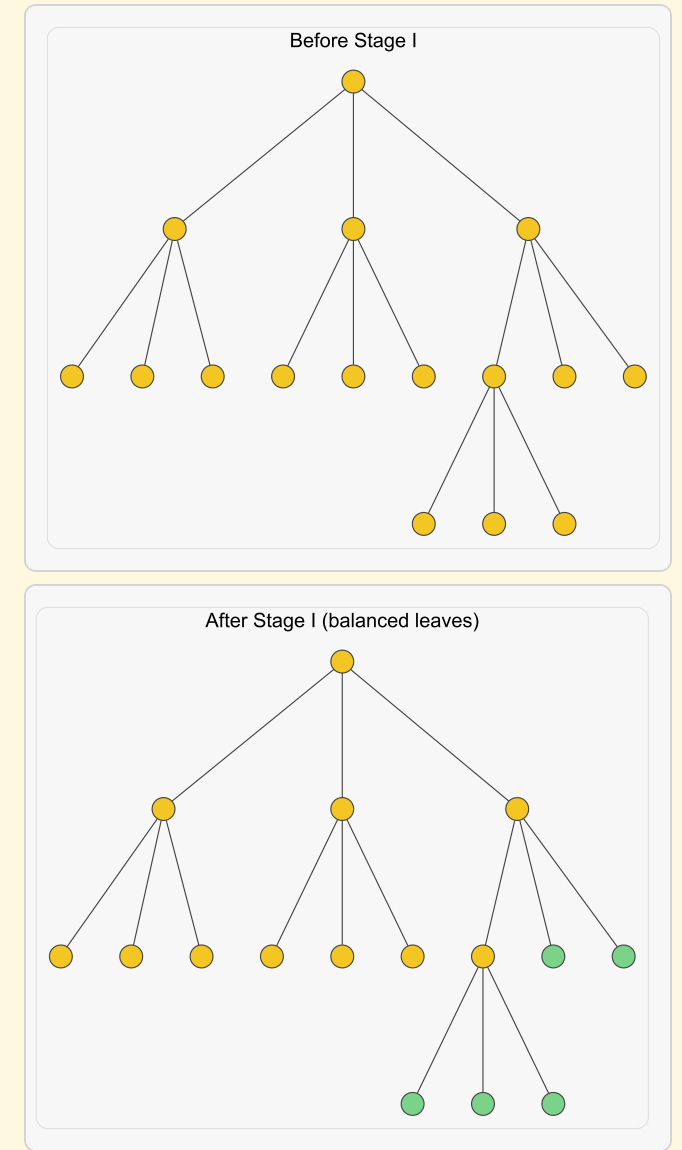
- Phase 1: Input generators
- Phase 2: Splitting strategies (Stage I-IV workflow)
- Phase 3: Hardcoded parameter extraction (offline replay)
- Phase 4: Output and visualization

Inputs And Tree Representation

- Input paths
 - checkpoint files via `GridPatchFileReader`
 - synthetic regular trees via `RegularGridGenerator`
- Data model: per-level arrays of tuples (`weight`, `subtree_id`)
- Leaf semantics handle adaptive trees in 2D and 3D

Stage I: Leaf Redistribution

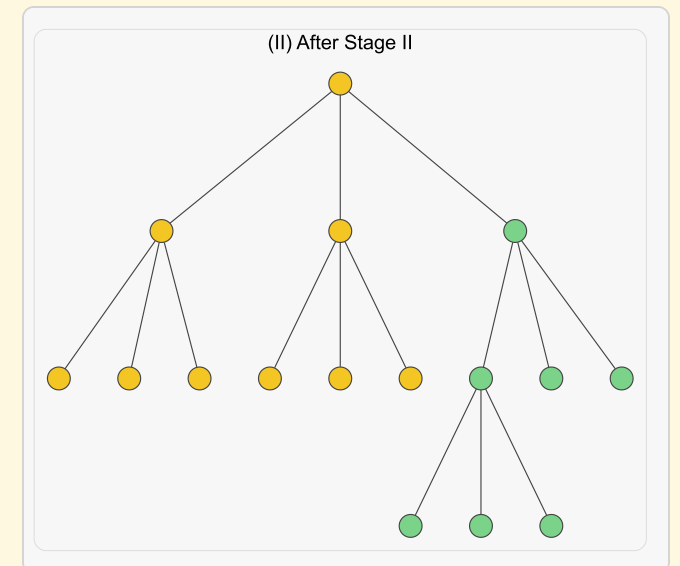
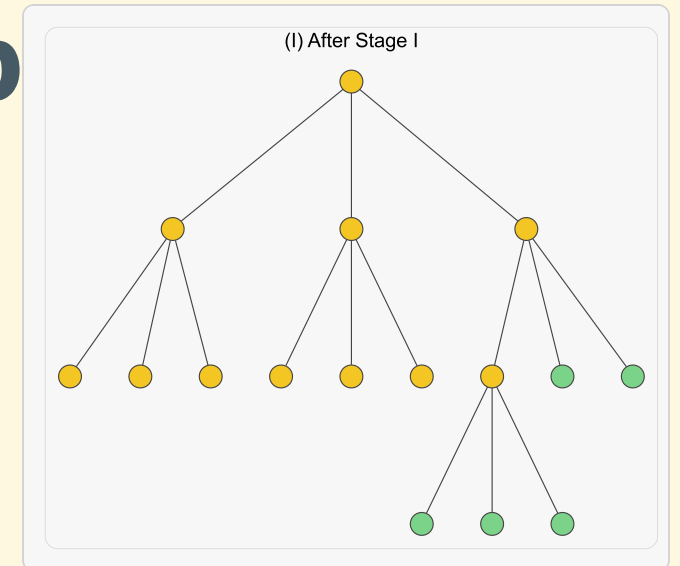
- Goal: **redistribute leaves** from subtree 0 across targets
- Traversals:
 - **bottom up** (per-level scanning over Peano SFC order on each level)
 - **top down** (depth-first-search scanning of levels, per-parent child traversal using Peano SFC order)
- Outcome: near-equal leaves per subtree (order changes only)
- Transition shown between diagrams: Stage I redistribution



Top: Before Stage I, Bottom: After Stage I

Stage II: Propagate Bottom-Up

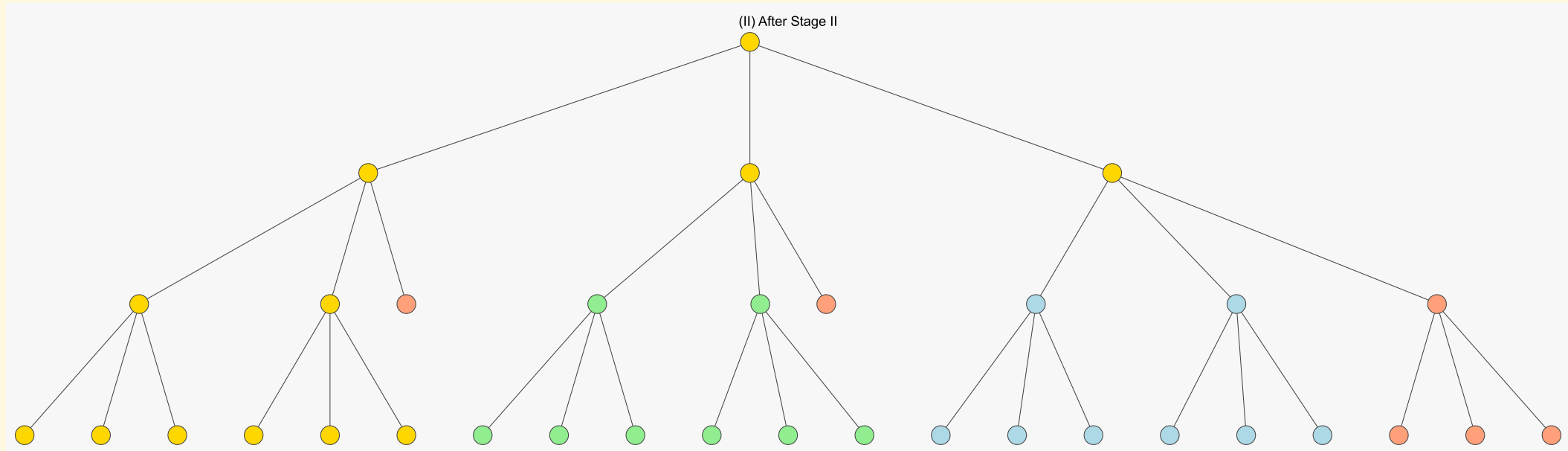
- Goal: **promote parent node** when all children agree on subtree assignment
- Traversal: **bottom-up** or **top-down** pass over the tree after Stage I
- Outcome: contiguous subtree boundaries instead of scattered leaf assignments
- Transition shown between diagrams: promote parent when children agree



Top: Stage I state, Bottom: Stage II propagated state

Stage III: Hierarchy Optimization

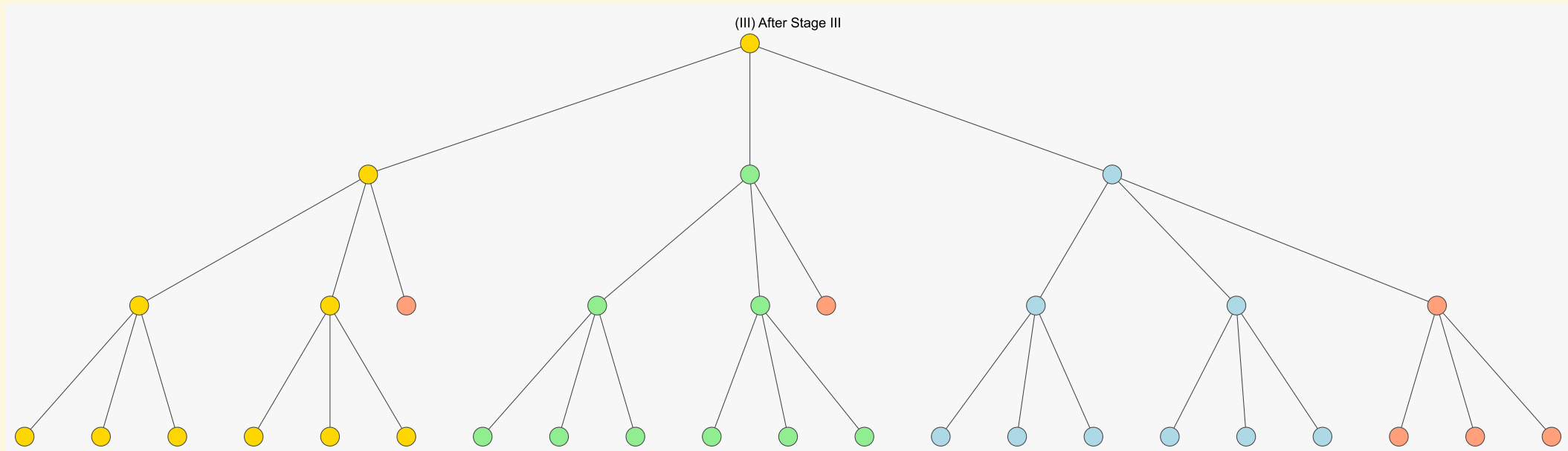
- Goal: **reduce split bookkeeping** by collapsing fully-owned subtrees
- Promoted subtree roots become **split boundaries**; their children are implicit
- Both stages preserve **validity constraints** from the API docs



- Stage II baseline shown here: compact Level-2 ownership after propagation

Stage III: Hierarchy Optimization (Result)

- After hierarchy lifting, dominant ownership is promoted to **coarser** levels
- In this example, subtree roots move from Level 2 to Level 1 where valid
- Same **leaf ownership**, fewer **split bookkeeping** points



- Transition from previous slide: Stage II to Stage III hierarchy lifting

Stage IV: Output Contract

- Extract arrays: `grid_sweeps`, `splitting_trees`, `number_of_cells`, `subtrees`
- **Subtree IDs** are logical; runtime maps them to destination **MPI ranks**
- Output directly configures the `Hardcoded` strategy

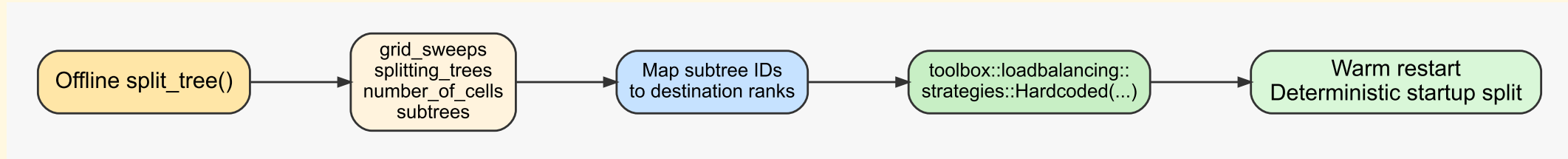
```
grid_sweeps, splitting_trees, number_of_cells, subtrees = split_strategy.split_tree(
    tree,
    number_of_subtrees=3,
    enable_propagate_bottom_up=True,
    enable_optimize_hierarchy=True,
    enable_extract_split_instructions=True,
) # creates subtrees {0,1,2}

# mapping subtrees to MPI ranks
destination_ranks = [
    tree_number % args.ranks for tree_number in subtrees
]

# Configure Hardcoded strategy
hardcoded_config = ( .. ) # skipped (splitting instruction) long string parameter for brevity
project.set_load_balancing(
    "toolbox::loadbalancing::strategies::Hardcoded",
    hardcoded_config
)

# Generate Peano4 project (C++ code with extracted optimal load balancing)
peano4_project.generate( .. ) # skipped long parameters for brevity
```

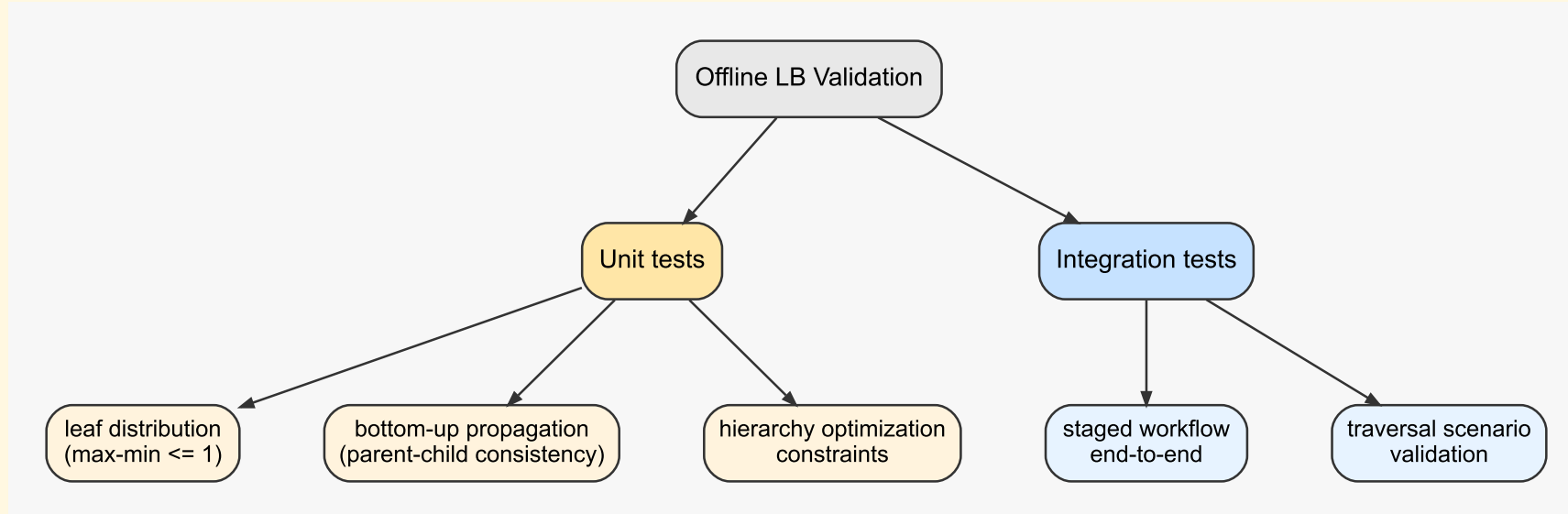
Runtime Bridge To C++ Replay



- Offline arrays become constructor arguments
- Startup split actions replay deterministically
- Runtime statistics expose balance quality

```
// Example constructor usage (C++)
toolbox::loadbalancing::strategies::Hardcoded(
    {2,4,6},           // gridSweeps
    {0,0,0},           // splittingTrees
    {9,81,1093},       // numberOfcells
    {1,1,1},           // destinationRanks (mapped from sub_trees)
    peano4::SplitInstruction::Mode::BottomUp
);
```

Development Testing

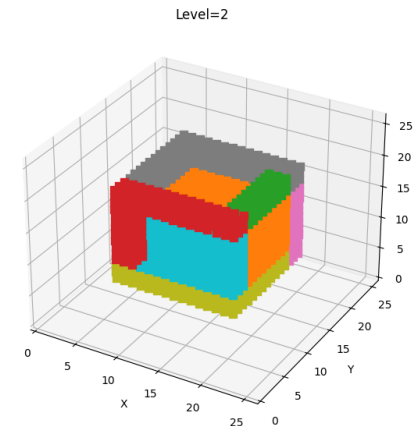
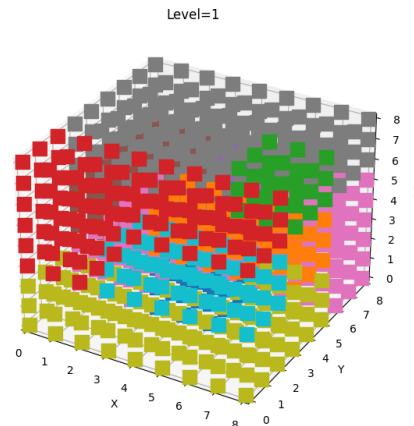
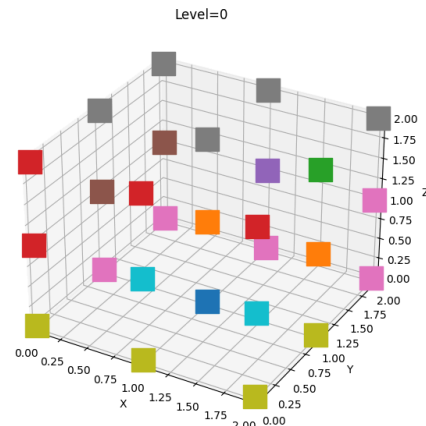


- Unit tests
 - leaf distribution and propagation invariants
 - hierarchy optimization constraints
- Integration tests
 - staged workflow end-to-end and traversal checks

Demo + Visuals (`OfflineDecomposition.py`)

- Before and after tree visuals from TreeVisualizer outputs
- Optional interactive HTML for exploration
- Demo script: load checkpoint tree → extract split instructions
- Generate C++ Peano emitting `Hardcoded` configuration (automatic from Python script, see tutorial)

```
# Analyze tree representing grid (synthetically generated or existing) and split it into subtrees
python3 OfflineDecomposition.py -f -l 3 -d 2 -s 2 -p # Build a regular 2D grid up to 3 levels, split into 2 subtrees, plot initial and resulting splitting
python3 OfflineDecomposition.py -f input/example -d 3 -s 4 -p # Read a 3D grid from the path, split into 4 subtrees, plot splitting
```



Peano and Load Balancing

Practical Walkthrough through a Tutorial (in progress)

1. Baseline run: no decomposition (`none`) to observe single-tree startup
2. Offline regular run: generate split instructions and print extracted arrays
3. Scale-out run: increase `--ranks` and `--trees` to map subtrees to ranks
4. Compare startup behavior and decomposition quality across the three runs
5. Build Peano project (C++ code)

```
# 1) Baseline (no offline decomposition)
python3 tutorial_offline_load_balancing_sbh.py --cell-size 1.8 --load-balancing-metric none

# 2) Offline decomposition over regular mesh
python3 tutorial_offline_load_balancing_sbh.py --trees 27 --ranks 1 --cell-size 1.8 --load-balancing-metric offline-regular-fine-grid

# 3) Same metric with multiple ranks (subtree -> rank mapping becomes visible)
python3 tutorial_offline_load_balancing_sbh.py --trees 27 --ranks 4 --cell-size 1.8 --load-balancing-metric offline-regular-fine-grid
```

Close And Next Action

- Offline processing improves startup decomposition quality
- The workflow is reproducible and test-backed
- Next step: apply the process to your own mesh snapshot

Thank you for attention!